



Mata Kuliah

2025

SISTEM BASIS DATA

Informatika Medis

Beberapa Perintah SQL Paling Penting

Beberapa Perintah SQL Paling Penting

SELECT - mengekstrak data dari database

UPDATE - memperbarui data dalam database

DELETE - menghapus data dari database

INSERT INTO - memasukkan data baru ke dalam database

CREATE DATABASE - membuat database baru

ALTER DATABASE - memodifikasi database

CREATE TABLE - membuat tabel baru

ALTER TABLE - mengubah tabel

DROP TABLE - menghapus tabel

CREATE INDEX - membuat indeks (kunci pencarian)

DROP INDEX - menghapus indeks



Mata Kuliah

2025

SISTEM BASIS DATA

Informatika Medis

Data Manipulation Language (DML)

DML – Select

SELECT digunakan untuk memilih data dari basis data.

Data yang dikembalikan disimpan dalam tabel hasil, yang disebut result-set.

```
SELECT column1, column2, ...  
FROM table_name;
```

```
SELECT * FROM table_name;
```

Contoh:

```
SELECT CustomerName, City, Country FROM Customers;
```

```
SELECT * FROM Customers;
```

DML – Select Distinct

SELECT DISTINCT digunakan untuk mengembalikan nilai yang berbeda saja.

Di dalam tabel, kolom sering kali berisi banyak nilai duplikat; dan terkadang Anda hanya ingin mencantumkan nilai yang berbeda (berbeda).

```
SELECT DISTINCT column1, column2, ...  
FROM table_name;
```

Contoh:

```
SELECT DISTINCT Country FROM Customers;
```

Apa Fungsi query dibawah? Ada yg bisa menjawab?

```
SELECT COUNT(DISTINCT Country) FROM Customers;
```

Clause – Where

WHERE digunakan untuk menyaring rekaman.

Digunakan untuk mengekstrak hanya rekaman yang memenuhi kondisi tertentu.

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

Contoh:

```
SELECT * FROM Customers
WHERE Country = 'Mexico';
```

```
SELECT * FROM Customers
WHERE CustomerID = 1;
```

Operator berikut dapat digunakan dalam WHERE klausa:

Operator	Description
=	Equal
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal
<>	Not equal. Note: In some versions of SQL this operator may be written as !=
BETWEEN	Between a certain range
LIKE	Search for a pattern
IN	To specify multiple possible values for a column

Operator - AND, OR dan NOT

WHERE dapat digabungkan dengan operator AND, OR, dan NOT

Operator AND dan OR digunakan untuk memfilter rekaman berdasarkan lebih dari satu kondisi:

- Operator AND menampilkan rekaman jika semua kondisi yang dipisahkan oleh AND adalah BENAR.
- Operator OR menampilkan rekaman jika salah satu kondisi yang dipisahkan oleh OR bernilai BENAR.

Operator NOT menampilkan rekaman jika kondisinya TIDAK BENAR.

AND

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 AND condition2 AND condition3 ...;
```

OR

```
SELECT column1, column2, ...  
FROM table_name  
WHERE condition1 OR condition2 OR condition3 ...;
```

NOT

```
SELECT column1, column2, ...  
FROM table_name  
WHERE NOT condition;
```

Contoh Operator AND, OR dan NOT

AND

Memilih semua bidang dari "Pelanggan" di mana negaranya adalah "Jerman" DAN kotanya adalah "Berlin"

```
SELECT * FROM Customers
WHERE Country = 'Germany' AND City = 'Berlin';
```

OR

Memilih semua bidang dari "Pelanggan" di mana kota adalah "Berlin" ATAU "Stuttgart"

```
SELECT * FROM Customers
WHERE City = 'Berlin' OR City = 'Stuttgart';
```

NOT

Memilih semua bidang dari "Pelanggan" yang negaranya BUKAN "Jerman"

```
SELECT * FROM Customers
WHERE NOT Country = 'Germany';
```

COMBINING (Menggabungkan Operator)

Memilih semua kolom dari "Pelanggan" yang negaranya adalah "Jerman" DAN kotanya harus "Berlin" ATAU "Stuttgart" (gunakan tanda kurung untuk membentuk ekspresi kompleks)

```
SELECT * FROM Customers
WHERE Country = 'Germany' AND (City = 'Berlin' OR City = 'Stuttgart');
```

Memilih semua kolom dari "Pelanggan" yang negaranya BUKAN "Jerman" dan BUKAN "AS"

```
SELECT * FROM Customers
WHERE NOT Country = 'Germany' AND NOT Country = 'USA';
```


ORDER BY Keyword

Kata ORDER BY digunakan untuk mengurutkan kumpulan hasil dalam urutan menaik atau menurun.

Kata ORDER BY kunci mengurutkan data dalam urutan menaik secara default. Untuk mengurutkan data dalam urutan menurun, gunakan DESC kata kunci .

```
SELECT column1, column2, ...  
FROM table_name  
ORDER BY column1, column2, ... ASC|DESC;
```

Contoh:

```
SELECT * FROM Customers  
ORDER BY Country;
```

```
SELECT * FROM Customers  
ORDER BY Country, CustomerName;
```

```
SELECT * FROM Customers  
ORDER BY Country DESC;
```

```
SELECT * FROM Customers  
ORDER BY Country ASC, CustomerName DESC;
```

DML – Insert Into

INSERT INTO digunakan untuk menyisipkan catatan baru dalam suatu tabel

Tentukan nama kolom dan nilai yang akan dimasukkan

```
INSERT INTO table_name (column1, column2, column3, ...)
VALUES (value1, value2, value3, ...);
```

ATAU

Jika menambahkan nilai untuk semua kolom tabel, Anda tidak perlu menentukan nama kolom dalam kueri SQL. Namun, pastikan urutan nilai berada dalam urutan yang sama dengan kolom dalam tabel

```
INSERT INTO table_name
VALUES (value1, value2, value3, ...);
```

Contoh:

```
INSERT INTO Customers (CustomerName, ContactName, Address, City, PostalCode, Country)
VALUES ('Cardinal', 'Tom B. Erichsen', 'Skagen 21', 'Stavanger', '4006', 'Norway');
```

```
INSERT INTO Customers (CustomerName, City, Country)
VALUES ('Cardinal', 'Stavanger', 'Norway');
```

NULL Values

Nilai NULL adalah bidang tanpa nilai.

Jika kolom dalam tabel bersifat opsional, Anda dapat menyisipkan rekaman baru atau memperbarui rekaman tanpa menambahkan nilai ke kolom ini. Kemudian, kolom akan disimpan dengan nilai NULL.

IS NULL (Menguji nilai kosong)

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NULL;
```

IS NOT NULL (Menguji nilai tidak kosong)

```
SELECT column_names  
FROM table_name  
WHERE column_name IS NOT NULL;
```

Contoh:

```
SELECT CustomerName, ContactName, Address  
FROM Customers  
WHERE Address IS NULL;
```

```
SELECT CustomerName, ContactName, Address  
FROM Customers  
WHERE Address IS NOT NULL;
```

Catatan: Nilai NULL berbeda dengan nilai nol atau kolom yang berisi spasi. Kolom dengan nilai NULL adalah kolom yang dibiarkan kosong selama pembuatan rekaman

DML – Update

UPDATE digunakan untuk mengubah catatan yang ada dalam suatu tabel

```
UPDATE table_name
SET column1 = value1, column2 = value2, ...
WHERE condition;
```

Contoh:

Memperbarui pelanggan pertama (CustomerID = 1) dengan kontak baru dan kota baru

```
UPDATE Customers
SET ContactName = 'Alfred Schmidt', City = 'Frankfurt'
WHERE CustomerID = 1;
```

Memperbarui PostalCode menjadi 00000 untuk semua catatan yang negaranya adalah "Meksiko"

```
UPDATE Customers
SET PostalCode = 00000
WHERE Country = 'Mexico';
```

Catatan: **Berhati-hatilah** saat memperbarui data dalam tabel! Perhatikan WHERE Clause dalam UPDATE pernyataan tersebut. WHERE Clause tersebut menentukan data mana saja yang harus diperbarui. Jika Anda menghilangkan WHERE Clause tersebut, semua data dalam tabel akan diperbarui!

DML – Delete

DELETE digunakan untuk menghapus rekaman yang ada pada tabel

```
DELETE FROM table_name WHERE condition;
```

Contoh:

Menghapus pelanggan "Alfreds Futterkiste" dari tabel "Pelanggan"

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

Menghapus semua baris dalam tabel tanpa menghapus tabel itu sendiri

```
DELETE FROM table_name;
```

Catatan: **Berhati-hatilah** saat menghapus data dalam tabel! Perhatikan WHERE Clause dalam DELETE pernyataan tersebut. WHERE Clause tersebut menentukan data mana saja yang harus dihapus. Jika Anda menghilangkan WHERE Clause tersebut, semua data dalam tabel akan dihapus!

DML – Delete

DELETE digunakan untuk menghapus rekaman yang ada pada tabel

```
DELETE FROM table_name WHERE condition;
```

Contoh:

Menghapus pelanggan "Alfreds Futterkiste" dari tabel "Pelanggan"

```
DELETE FROM Customers WHERE CustomerName='Alfreds Futterkiste';
```

Menghapus semua baris dalam tabel tanpa menghapus tabel itu sendiri

```
DELETE FROM table_name;
```

Catatan: **Berhati-hatilah** saat menghapus data dalam tabel! Perhatikan WHERE Clause dalam DELETE pernyataan tersebut. WHERE Clause tersebut menentukan data mana saja yang harus dihapus. Jika Anda menghilangkan WHERE Clause tersebut, semua data dalam tabel akan dihapus!

Clause – Limit

LIMIT digunakan untuk menentukan jumlah rekaman yang akan dikembalikan.

LIMIT ini berguna pada tabel besar dengan ribuan catatan. Mengembalikan sejumlah besar catatan dapat memengaruhi kinerja.

```
SELECT column_name(s)
FROM table_name
WHERE condition
LIMIT number;
```

Contoh:

Memilih tiga catatan pertama dari tabel "Pelanggan"

```
SELECT * FROM Customers
LIMIT 3;
```

Hanya mengembalikan 3 rekaman, mulai pada rekaman 4 (OFFSET 3)

```
SELECT * FROM Customers
LIMIT 3 OFFSET 3;
```

Memilih tiga catatan pertama dari tabel "Pelanggan", di mana negaranya adalah "Jerman"

```
SELECT * FROM Customers
WHERE Country='Germany'
LIMIT 3;
```

Mengurutkan pelanggan berdasarkan negara sebelum mengembalikan tiga catatan pertama dari pilihan

```
SELECT * FROM Customers
ORDER BY Country
LIMIT 3;
```

Functions - Min & Max

MIN() mengembalikan nilai terkecil dari kolom yang dipilih.

MAX() mengembalikan nilai terbesar dari kolom yang dipilih.

```
SELECT MIN(column_name)
FROM table_name
WHERE condition;
```

```
SELECT MAX(column_name)
FROM table_name
WHERE condition;
```

Contoh:

Menemukan harga produk termurah

```
SELECT MIN(Price) AS SmallestPrice
FROM Products;
```

Menemukan harga produk yang paling mahal

```
SELECT MAX(Price) AS LargestPrice
FROM Products;
```


Functions – COUNT, AVG, & SUM

COUNT() mengembalikan jumlah baris yang cocok dengan kriteria yang ditentukan

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

AVG() mengembalikan nilai rata-rata kolom numerik

```
SELECT AVG(column_name)
FROM table_name
WHERE condition;
```

SUM() mengembalikan jumlah total kolom numerik

```
SELECT SUM(column_name)
FROM table_name
WHERE condition;
```

Contoh:

Menemukan jumlah produk

```
SELECT COUNT(ProductID)
FROM Products;
```

Menemukan harga rata-rata semua produk

```
SELECT AVG(Price)
FROM Products;
```

Menemukan jumlah kolom "Quantity" dalam tabel "OrderDetails"

```
SELECT SUM(Quantity)
FROM OrderDetails;
```

Operator – Like

Operator LIKE digunakan dalam WHERE Clause untuk mencari pola tertentu dalam kolom.

Ada dua karakter pengganti yang sering digunakan bersama dengan LIKE operator:

- Tanda persen (%) mewakili nol, satu, atau beberapa karakter
- Tanda garis bawah (_) mewakili satu karakter tunggal

Tanda persen dan garis bawah juga dapat digunakan dalam kombinasi

```
SELECT column1, column2, ...
FROM table_name
WHERE columnN LIKE pattern;
```

LIKE Operator	Description
WHERE CustomerName LIKE 'a%'	Finds any values that start with "a"
WHERE CustomerName LIKE '%a'	Finds any values that end with "a"
WHERE CustomerName LIKE '%or%'	Finds any values that have "or" in any position
WHERE CustomerName LIKE '_r%'	Finds any values that have "r" in the second position
WHERE CustomerName LIKE 'a_%'	Finds any values that start with "a" and are at least 2 characters in length
WHERE CustomerName LIKE 'a__%'	Finds any values that start with "a" and are at least 3 characters in length
WHERE ContactName LIKE 'a%o'	Finds any values that start with "a" and ends with "o"

Contoh Operator Like

Memilih semua pelanggan dengan CustomerName yang dimulai dengan "a"

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';
```

Memilih semua pelanggan dengan CustomerName yang diakhiri dengan "a"

```
SELECT * FROM Customers
WHERE CustomerName LIKE '%a';
```

Memilih semua pelanggan dengan CustomerName yang memiliki "atau" di posisi mana pun

```
SELECT * FROM Customers
WHERE CustomerName LIKE '%or%';
```

Catatan: Anda juga dapat menggabungkan sejumlah kondisi menggunakan operator AND atau OR.

Memilih semua pelanggan dengan CustomerName yang memiliki "r" di posisi kedua

```
SELECT * FROM Customers
WHERE CustomerName LIKE '_r%';
```

Memilih semua pelanggan dengan CustomerName yang dimulai dengan "a" dan panjangnya minimal 3 karakter

```
SELECT * FROM Customers
WHERE CustomerName LIKE 'a__%';
```

Memilih semua pelanggan dengan ContactName yang dimulai dengan "a" dan diakhiri dengan "o"

```
SELECT * FROM Customers
WHERE ContactName LIKE 'a%o';
```

Memilih semua pelanggan dengan CustomerName yang TIDAK dimulai dengan "a"

```
SELECT * FROM Customers
WHERE CustomerName NOT LIKE 'a%';
```

Characters - Wildcards

Karakter wildcard digunakan untuk mengganti satu atau beberapa karakter dalam string.

Karakter wildcard digunakan dengan operator LIKE. Operator LIKE digunakan dalam Clause WHERE untuk mencari pola tertentu dalam kolom.

Symbol	Description	Example
%	Represents zero or more characters	bl% finds bl, black, blue, and blob
_	Represents a single character	h_t finds hot, hat, and hit

Contoh:

Memilih semua pelanggan dengan Kota yang berisi pola "es"

```
SELECT * FROM Customers
WHERE City LIKE '%es%';
```

Memilih semua pelanggan dengan Kota yang dimulai dengan "L", diikuti oleh karakter apa pun, diikuti oleh "n", diikuti oleh karakter apa pun, diikuti oleh "on"

```
SELECT * FROM Customers
WHERE City LIKE 'L_n_on';
```

Operator - IN

IN memungkinkan Anda untuk menentukan beberapa nilai dalam sebuah **WHERE** Clause
IN adalah singkatan untuk beberapa **OR** kondisi.

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (value1, value2, ...);
```

ATAU

```
SELECT column_name(s)
FROM table_name
WHERE column_name IN (SELECT STATEMENT);
```

Contoh:

Memilih semua pelanggan yang berlokasi di "Jerman", "Prancis", atau "Inggris"

```
SELECT * FROM Customers
WHERE Country IN ('Germany', 'France', 'UK');
```

Memilih semua pelanggan yang berasal dari negara yang sama dengan pemasok

```
SELECT * FROM Customers
WHERE Country IN (SELECT Country FROM Suppliers);
```

Memilih semua pelanggan yang **TIDAK** berlokasi di "Jerman", "Prancis", atau "Inggris"

```
SELECT * FROM Customers
WHERE Country NOT IN ('Germany', 'France', 'UK');
```

Operator - BETWEEN

Operator BETWEEN memilih nilai dalam rentang tertentu. Nilai dapat berupa angka, teks, atau tanggal.

Operator BETWEEN bersifat inklusif: nilai awal dan akhir disertakan.

```
SELECT column_name(s)
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Contoh:

Memilih semua produk dengan harga antara 10 dan 20

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20;
```

Menampilkan produk di luar rentang contoh sebelumnya

```
SELECT * FROM Products
WHERE Price NOT BETWEEN 10 AND 20;
```

Memilih semua produk dengan harga antara 10 dan 20. Selain itu, jangan tampilkan produk dengan CategoryID 1,2, atau 3

```
SELECT * FROM Products
WHERE Price BETWEEN 10 AND 20
AND CategoryID NOT IN (1,2,3);
```

Memilih semua produk dengan ProductName antara "Carnarvon Tigers" dan "Mozzarella di Giovanni"

```
SELECT * FROM Products
WHERE ProductName BETWEEN 'Carnarvon Tigers' AND 'Mozzarella di Giovanni'
ORDER BY ProductName;
```

Aliases

Alias digunakan untuk memberi nama sementara pada tabel atau kolom dalam tabel

Alias sering digunakan untuk membuat nama kolom lebih mudah dibaca

Alias hanya ada selama kueri tersebut berlangsung

Alias dibuat dengan AS kata kunci.

```
SELECT column_name AS alias_name
FROM table_name;
```

```
SELECT column_name(s)
FROM table_name AS alias_name;
```

Contoh:

Membuat dua alias, satu untuk kolom CustomerName dan satu untuk kolom ContactName

```
SELECT CustomerName AS Customer, ContactName AS "Contact Person"
FROM Customers;
```

Membuat alias bernama "Alamat" yang menggabungkan empat kolom (Alamat, Kodepos, Kota dan Negara)

```
SELECT CustomerName, CONCAT_WS(' ', Address, PostalCode, City, Country) AS Address
FROM Customers;
```

Memilih semua pesanan dari pelanggan dengan CustomerID=4 (Around the Horn). Kami menggunakan tabel "Customers" dan "Orders", dan memberi mereka alias tabel "c" dan "o" secara berurutan (menggunakan alias untuk mempersingkat SQL)

```
SELECT o.OrderID, o.OrderDate, c.CustomerName
FROM Customers AS c, Orders AS o
WHERE c.CustomerName='Around the Horn' AND c.CustomerID=o.CustomerID;
```

Sumber

- Umar Zaky, S.Kom., M.Cs.
- W3schools



Terima Kasih